

Accelerated Mobile Video Processing

Project Status Report

Nitish Kumar (2025MCS2100) Salil Gujar (2025MCS2106)

Where We Stand

Both Phase 1 and Phase 2 are done as per our checkpoints — and just to clarify, these checkpoints were made by us as part of our own project plan. We wrote them down at the start to keep ourselves on track, so hitting both of them feels good.

At this point the app is fully working on a real Android phone. It opens the camera, processes each frame through a Sobel edge detection filter written in C++, and shows the output live on screen along with performance numbers. We have two processing modes — a simple scalar baseline and a faster NEON SIMD version — and you can switch between them on the fly with a button. The big result so far is that the NEON path is **6.2× faster** on the Sobel step than the scalar one. Phase 3 (GPU) and Phase 4 (final tuning) are still ahead of us.

1. What the Project Is About

The idea is straightforward. We take live frames from the phone camera, run a Sobel edge detection filter on each frame using native C++ code (not Java/Kotlin), and draw the result on screen in real time. The filter highlights edges in the scene — things like chair outlines, cables, table corners. Then we keep making that filter faster by changing how the code works internally, and we measure the improvement at each step.

The four steps we planned are: (1) a basic scalar C implementation to get a working baseline, (2) an ARM NEON SIMD version that processes 16 pixels at once instead of one, (3) a GPU compute shader using OpenGL ES, and (4) a hybrid mode that uses both CPU and GPU together. We picked Sobel because the math is simple and it is easy to verify — you can check any output pixel against a reference without needing its neighbours.

We built everything on Android API 26+, using Kotlin and Jetpack Compose for the UI, and the Android NDK for the native C++ side. The camera runs at 1280×720 resolution. All the heavy processing happens in native code and Kotlin just handles drawing and the stats overlay.

2. Phase 1 — Getting the Baseline Working

2.1 Stage 1 — Setting up the project

We created the Android project, enabled the NDK, and wrote a tiny JNI function called `nativeGetStatus()` just to confirm that the Kotlin-to-C++ bridge was working. Nothing fancy here — just making sure the toolchain was set up correctly before writing any real code.

2.2 Stage 2 — Getting camera frames

We used the Camera2 API to open the rear camera and lock it to a 30 FPS target. Each frame comes in as raw YUV_420_888 data and lands in an `ImageReader` callback running on a background thread. We also added a check that compares hardware timestamps between consecutive frames — if the gap goes above 40 ms it means frames are being dropped, and we log a warning.

2.3 Stage 3 — Converting YUV to RGBA

The camera gives us YUV colour format but we need RGBA to work with. So we wrote `nativeYuvToRgba()` which reads the Y, U and V planes with their actual row strides (since they do not always sit back-to-back in memory) and converts each pixel using the standard BT.601 formula in fixed-point arithmetic. We keep this inside a JNI critical section so the Android garbage collector cannot interrupt it mid-frame.

2.4 Stage 4 — The scalar Sobel filter

The Sobel filter works by running two small 3×3 kernels over each pixel — one detects horizontal edges (G_x) and one detects vertical edges (G_y). We compute the edge strength as $(|G_x| + |G_y|)/2$, which is an approximation that avoids the square root and is visually almost identical to the proper Euclidean norm. For pixels on the border of the image we clamp the index instead of going out of bounds. This gave us a working, correct baseline to measure from.

2.5 Stage 5 — Displaying the result

The output buffer gets wrapped into a `Bitmap`, rotated 90 degrees to match the phone's portrait orientation, and passed to a `Compose Image` composable to render on screen. We recycle the previous bitmap immediately to avoid building up memory.

2.6 Stage 6 — The live stats overlay

On top of the camera output we draw a small text overlay that updates every frame. Here is what each number on the screen means:

- **MODE** — tells you which path is running: Baseline (the scalar C version) or SIMD (the NEON version).
- **FPS** — frames per second, calculated as a rolling average over the last 30 frames using hardware timestamps from the camera.
- **Intv** — the time gap between two consecutive frames in milliseconds, read directly from the camera hardware timestamps. At real 30 FPS this should be about 33 ms. If it reads 66 ms it means every second frame is being dropped.
- **JNI** — total time our native code takes per frame, including both the colour conversion and the Sobel filter together.
- **Conv** — just the YUV-to-RGBA conversion time on its own.
- **Sobel** — just the Sobel filter time on its own. This is the number we care most about because it is what we are optimising.
- **E2E** — end-to-end time from when a frame arrives at the callback to when the bitmap is ready for the display.
- **SIMD: PASS / FAIL** — result of our correctness check. Every frame, we run the NEON output through a plain scalar re-computation and compare pixel by pixel. If

any pixel is off by more than 1 it prints FAIL, otherwise PASS. So far every single frame passes.

- **Base / NEON** — shown only in SIMD mode. These are the stored Sobel times from each mode so you can compare them side by side even though only one runs at a time.
- **Speedup** — the ratio Base/NEON. We explain exactly how this is computed in Section 6.
- **CPU** — overall device CPU load read from `/proc/stat`, updated once per second.

3. Phase 2 — Making It Faster with NEON

3.1 Stage 1 — Warming up with NEON

Before touching the Sobel filter, we wrote a simpler function — `nativeGrayscaleNeon()` — that converts RGBA pixels to greyscale using NEON vector registers. It processes 16 pixels at a time using `uint8x16_t` registers, and uses integer luma weights that add up to 256 so we can shift right instead of doing a real divide. This was mainly to check that our NEON build flags were correct and that we knew how to write NEON code before tackling the more complex Sobel kernel.

3.2 Stage 2 — NEON Sobel

The main optimisation. `nativeSobelNeon()` processes 16 pixel columns per loop iteration instead of one. Here is what each NEON call does:

- `vld4q_u8` — loads 64 bytes from memory and automatically separates the four RGBA channels into four separate 16-element vectors. We do this for all eight neighbour pixel positions around the current column.
- `vaddl_u8` — widens values from 8-bit to 16-bit before we add them together, so we do not get overflow.
- `vabdq_u16` — computes the absolute difference to get $|G_x|$ and $|G_y|$ without any branching.
- `vqshrn_n_u16` — shifts right by one (divides by 2), narrows back to 8-bit, and clamps to 255 if it overflows — all in a single instruction.
- `vst4q_u8` — re-interleaves the four channels and writes 64 bytes back to the output buffer.

Any leftover columns at the right edge of the frame (since the width may not be a perfect multiple of 16) fall through to the scalar path, so the full-frame output is always consistent.

3.3 Stage 3 — Checking correctness

We added `nativeVerifySobelCorrectness()` which runs the NEON path on a frame and then runs the scalar formula on the same frame and compares every interior pixel. If any pixel differs by more than 1 it flags FAIL. In all our testing every frame comes back PASS with zero mismatches, which gives us confidence the NEON code is doing the right thing.

3.4 Stage 4 — Switching between modes

There is a button at the bottom of the screen that swaps between scalar and NEON mode. The switch happens on the very next incoming frame with no restart needed. This is

useful for demos because you can see the FPS change in real time as you tap the button.

3.5 Stage 5 — Showing the speedup

Once you have run both modes at least once, the dashboard shows the last measured Sobel time for each mode and the speedup ratio. On our test phone the scalar path takes around 31 ms for the Sobel step and the NEON path takes 5 ms, giving a speedup of **6.2×**. The full numbers and the calculation are in Section 6.

4. Phases 3 and 4 — What Is Still Left

Phase 3 is the GPU part. The plan is to set up a headless EGL context, write a GLSL compute shader that runs the Sobel filter on the GPU, and also build a Hybrid mode where the frame is split between the NEON thread and the GPU shader running at the same time. Neither of us has written any GPU code for this project yet — Phase 3 starts after this submission.

Phase 4 is about polishing everything: proper latency profiling, reusing frame buffers instead of allocating new ones, adding NEON prefetch hints, tuning GPU workgroup sizes, and running the app for 30 minutes straight to check for memory leaks or crashes. Also the Bitmap rotation still creates a new object every frame which we know is wasteful — that gets fixed in Phase 4.

5. GitHub Commit History

We have been committing steadily since 22 February 2026. The full commit history screenshot is included at the end of this report (see Figure 4).

6. The Numbers and How We Calculate Speedup

The numbers below are taken from the screenshots below. One thing worth knowing is that all these values fluctuate live on screen — FPS is a rolling 30-frame average, latency numbers update every frame, and the CPU reading updates once a second. So what we see in a screenshot is a snapshot at one particular instant. The numbers shift a little from moment to moment, but the general picture stays consistent: baseline mode sits around 20–22 FPS and SIMD mode holds steady around 30 FPS. The table shows the values captured in our screenshots.

Metric	Baseline mode	SIMD (NEON) mode
FPS	21.9	30.2
Frame interval	66 ms	33 ms
JNI total latency	33 ms	11 ms
YUV conversion	4 ms	6 ms
Sobel filter	29 ms	5 ms
End-to-end latency	45 ms	28 ms
SIMD correctness check	PASS	PASS
CPU usage	0.0%	0.0%
Stored baseline Sobel (Base)	—	31 ms
NEON Sobel (NEON)	—	5 ms
Speedup (Sobel)	—	6.2×

Why FPS drops in Baseline mode

The camera is set to target 30 FPS, so it should deliver a new frame every $1000/30 \approx 33$ ms. But in Baseline mode the Sobel step alone takes 29 ms, and the total JNI time (conversion + Sobel) reaches 33 ms. That already fills one entire frame period, so the app cannot finish processing before the next frame arrives. The result is that every other frame gets dropped. You can see this in the frame interval reading — it shows 66 ms, which is exactly two frame periods, and the effective FPS falls to 21.9.

This is not a surprise. This is exactly the problem we were trying to show before fixing it with NEON.

After switching to NEON, the Sobel step drops to 5 ms and total JNI time comes down to 11 ms. The pipeline now finishes well before the next frame arrives and the interval goes back to 33 ms with a clean 30.2 FPS.

How we calculate the speedup

The app keeps the last measured Sobel time for each mode stored separately. When you are in SIMD mode, the overlay shows both values at once: “Base: 31 ms” (the last time Sobel ran in scalar mode) and “NEON: 5 ms” (the current NEON time). The speedup is just the ratio:

$$\text{Speedup} = \frac{\text{Baseline Sobel time}}{\text{NEON Sobel time}} = \frac{31 \text{ ms}}{5 \text{ ms}} = 6.2\times$$

You might notice the baseline screenshot shows Sobel as 29 ms while the SIMD screen stores it as 31 ms. That is completely normal — the numbers fluctuate slightly every frame and these two screenshots were taken at different moments. The speedup on screen is always computed live from whatever the two stored values happen to be at that instant, so $31/5$ gives the $6.2\times$ shown there. If you watch the app running you will see the speedup number nudge up and down a little as fresh measurements come in.

To put it simply: the NEON version finishes the same Sobel computation more than six times faster than the scalar version, which is enough to unlock proper 30 FPS video processing on a mid-range phone.

Repository: https://github.com/salil-123/Design_Document

Appendix A — GitHub Commit History Screenshot

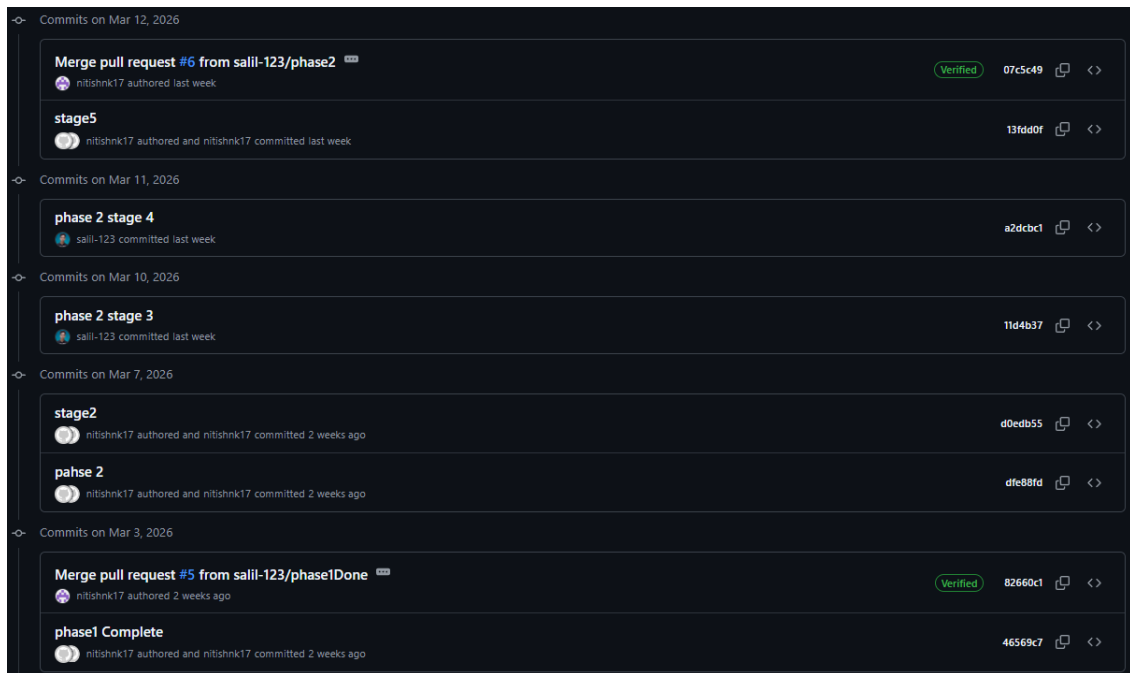


Figure 1: Commit History Screenshot 1 (Most Recent)



Figure 2: Commit History Screenshot 2

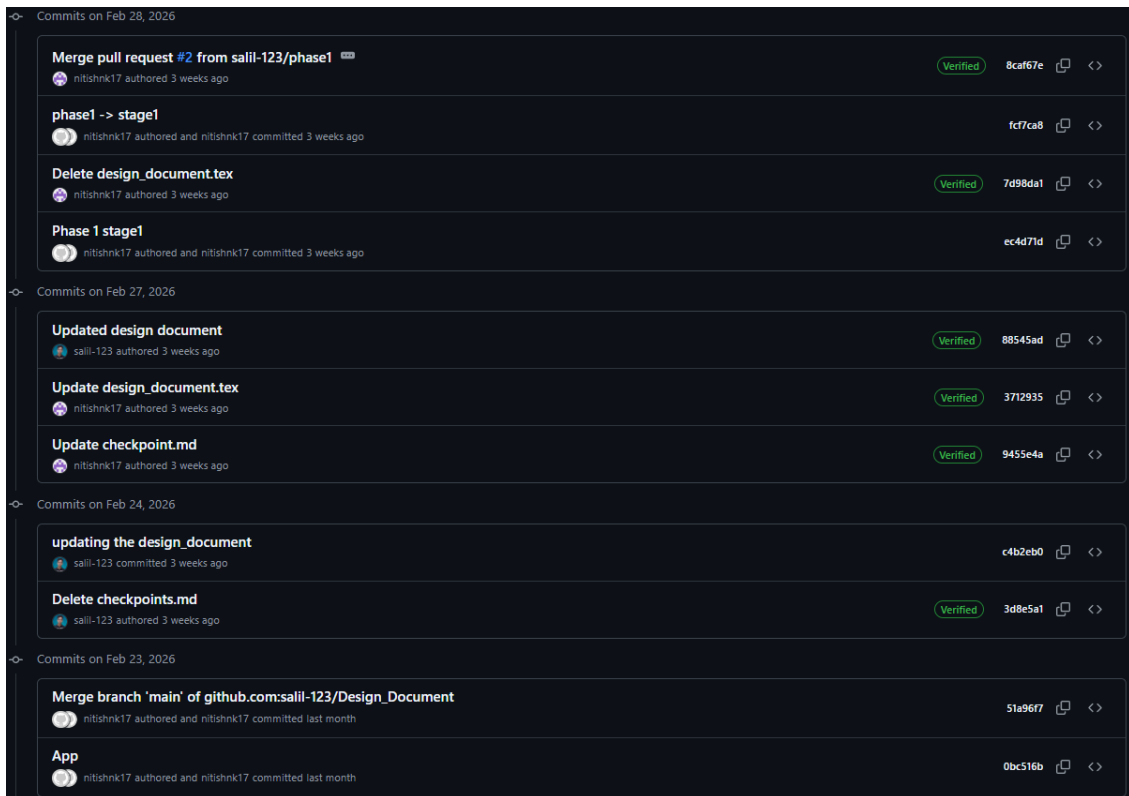


Figure 3: Commit History Screenshot 3

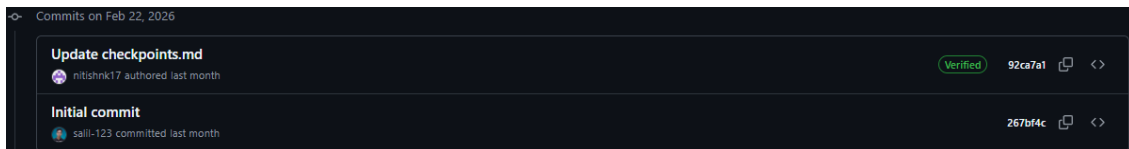


Figure 4: Commit History Screenshot 4 (Earliest)

Appendix B — App Screenshots from the Device

These screenshots were taken from the actual test phone. Keep in mind that all the numbers on screen update live every frame, so what you see here is just a snapshot — the values shift slightly from moment to moment as the rolling averages recalculate.

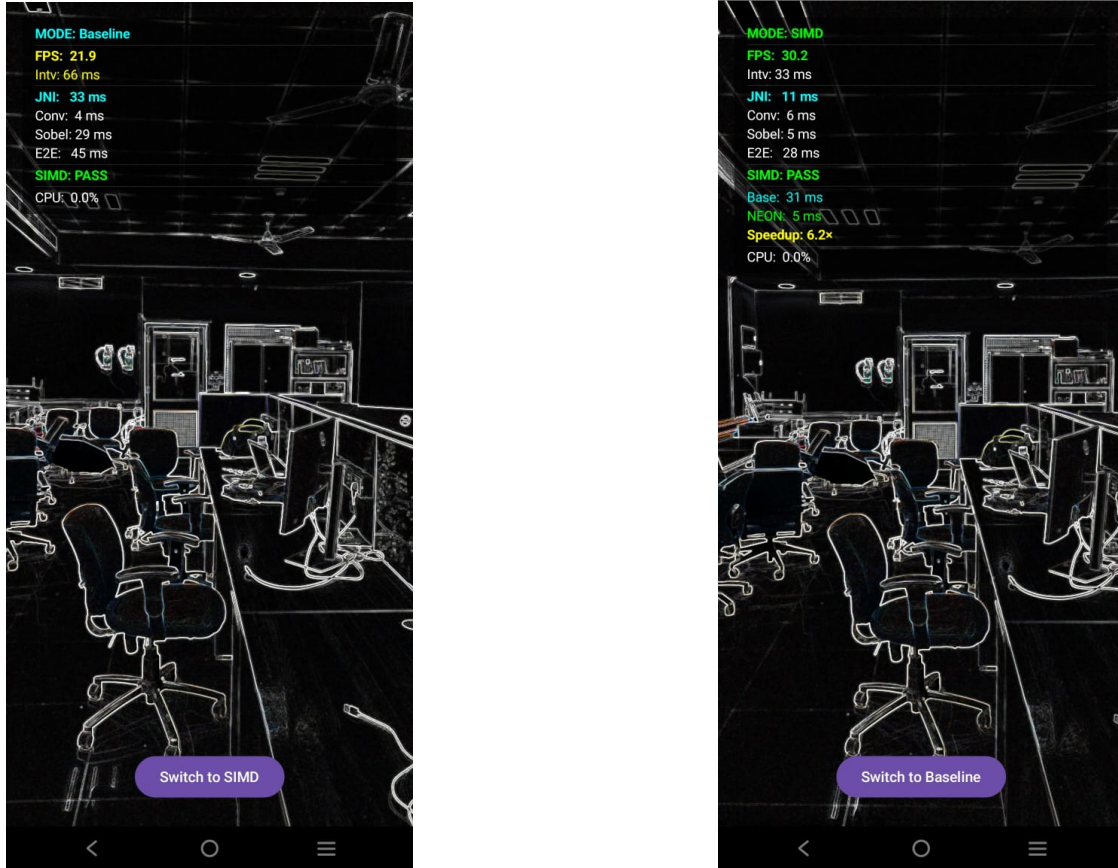


Figure 5: Left: Baseline (scalar) mode. Right: SIMD (NEON) mode. These are single-frame snapshots — the numbers on screen fluctuate slightly every frame as the rolling averages update, so treat them as representative readings rather than exact fixed values.